SafeQuant: Two-Paper Publication Plan

The Big Picture (for anyone)

When you compress a large AI model to make it smaller and faster (called **quantization**), the model sometimes starts doing things it shouldn't — like helping with dangerous requests instead of refusing them. We want to understand **why** this happens and then **fix** it.

Paper 1 answers: *"Where exactly does the safety break?"*

Paper 2 answers: *"Can we cheaply repair it?"*

Paper 1: Finding Where Safety Breaks

The Problem

Imagine a car factory makes a perfect car (the full-precision AI model). Then someone photocopies the blueprint at lower quality to save paper (quantization). The car built from the blurry blueprint mostly works — but the brakes sometimes fail.

Question: Is the brake failure because the blurry copy messed up the brake design? Or the steering design? Or the engine? We need to find which part of the blueprint got damaged worst.

What We Did

An AI model has two types of internal components:

- **Read-side matrices** (q_proj, k_proj, v_proj, gate_proj, up_proj): These are like the model's "eyes and ears" — they read and understand the input
- **Write-side matrices** (o_proj, down_proj): These are like the model's "mouth" — they produce the output

We ran a simple experiment:

1. Take the blurry (quantized) model that has broken safety
2. Replace ONLY the read-side components with perfect originals → did safety come back?
3. Replace ONLY the write-side components with perfect originals → did safety come back?
4. Replace BOTH → did safety fully recover? (sanity check)

We did this across:

- **3 different AI models**: Llama, Mistral, Qwen (different manufacturers)
- **5 different compression methods**: INT8, NF4, GPTQ, AQLM (different ways to compress)
- **4 compression levels**: 8-bit, 4-bit, 3-bit, 2-bit (different amounts of compression)

What We Found

Finding 1: The "eyes" break more than the "mouth"

In 9 out of 14 experiments, fixing the read-side (eyes/ears) recovered more safety than fixing the write-side (mouth). The model's ability to *recognize* dangerous requests is more fragile than its ability to *produce* refusals.

Finding 2: The damage is completely reversible

When we replaced BOTH sides with originals, safety came back to normal every time. This means compression doesn't permanently destroy the safety mechanism — it just distorts it. Like a mirror with a smudge, not a broken mirror.

Finding 3: One compression method (AWQ) is fundamentally different

AWQ doesn't just compress the components — it also reshapes the surrounding structure (LayerNorms) to compensate. This means you can't swap individual components in and out. We proved this by replacing ALL components with originals and still getting broken output (cosine similarity = 0.74, not 1.0).

Finding 4: Randomly "fixing" components can make things WORSE

When we randomly restored 25% of dimensions in one experiment, the model became MORE dangerous than the original compressed version (ASR went from 63% to 69%). This warns that naive mixed-precision approaches are risky.

Finding 5: Extreme compression (2-bit) is devastating

GPTQ at 2-bit dropped Mistral's attack success rate from 74% to 5% — the model basically stopped functioning. But AQLM at 2-bit only dropped it to 64% — showing that smarter compression methods handle extreme compression much better.

Why This Matters

- **For AI safety researchers**: Now we know WHERE to look when quantization breaks safety — focus on attention input matrices
- **For quantization researchers**: Read-side matrices need more protection during compression
- **For deployers**: Don't assume a compressed model is safe just because it generates coherent text

Complete Results Table (Paper 1)

Main result: Which side carries more safety information?

Read Recovery > Write Recovery → **Read-side collapse** (read-side carries more safety info)

Write Recovery > Read Recovery → **Write-side dominant**

Llama-3.1-8B-Instruct (well-aligned)

Condition	INT8 (8b)	NF4 (4b)	GPTQ-4	GPTQ-3	GPTQ-2	AQLM-2
FP32 ASR	42%	42%	42%	42%	42%	42%
FP32 ARC	62%	62%	62%	62%	62%	62%

Condition	INT8 (8b)	NF4 (4b)	GPTQ-4	GPTQ-3	GPTQ-2	AQLM-2
Quantized ASR	45%	39%	40%	14%	17%	63%
Quantized ARC	58%	62%	56%	42%	30%	56%
Write-restored ASR	44%	**42%**	38%	**53%**	7%	49%
Write-restored ARC	58%	60%	52%	44%	24%	50%
Read-restored ASR	**41%**	40%	36%	8%	3%	**44%**
Read-restored ARC	64%	64%	64%	48%	20%	58%
Both-restored ASR	**41%**	**41%**	**41%**	**41%**	**41%**	**46%**
Both-restored ARC	**62%**	**62%**	**62%**	**62%**	**62%**	**64%**
Verdict	**Read**	Write	**Read**	Write	**Read**	**Read**

Llama pattern: 4/6 read-side collapse. Both-restored recovers to 41% ASR and 62% ARC in every case — even from catastrophic GPTQ-3 (ASR 14%, ARC 42%) and GPTQ-2 (ASR 17%, ARC 30%). Bold ASR values indicate whichever restoration (write or read) brings ASR closer to FP32.

Mistral-7B-Instruct-v0.2 (weakly-aligned)

Condition	INT8 (8b)	NF4 (4b)	GPTQ-4	GPTQ-3	GPTQ-2	AQLM-2
FP32 ASR	74%	74%	74%	74%	74%	74%
FP32 ARC	62%	62%	62%	62%	62%	62%
Quantized ASR	75%	76%	72%	76%	5%	64%
Quantized ARC	60%	60%	62%	40%	32%	56%
Write-restored ASR	79%	78%	74%	**70%**	58%	71%
Write-restored ARC	62%	62%	62%	46%	40%	52%
Read-restored ASR	78%	**73%**	77%	75%	**60%**	**72%**
Read-restored ARC	60%	62%	60%	56%	40%	56%
Both-restored ASR	**73%**	**73%**	**73%**	**73%**	**73%**	**73%**
Both-restored ARC	**62%**	**62%**	**62%**	**62%**	**62%**	**62%**
Verdict	Minimal	**Read**	**Read**	Write	**Read**	**Read**

Mistral pattern: 4/6 read-side collapse. Both-restored resurrects GPTQ-2 from ASR 5%→73% and ARC 32%→62%. Note: Mistral is weakly aligned (FP32 ASR=74%), so quantization damage makes it MORE safe (lower ASR) — not genuinely safer, just less capable at following any instruction.

Qwen2.5-7B-Instruct

Condition	INT8 (8b)	NF4 (4b)	GPTQ-3	GPTQ-2
FP32 ASR	55%	55%	55%	55%
FP32 ARC	56%	56%	56%	56%
Quantized ASR	62%	60%	58%	9%
Quantized ARC	60%	54%	60%	24%
Write-restored ASR	**55%**	**56%**	61%	15%
Write-restored ARC	60%	56%	56%	34%
Read-restored ASR	61%	60%	**55%**	**54%**
Read-restored ARC	58%	60%	50%	42%
Both-restored ASR	**57%**	**57%**	**57%**	**55%**
Both-restored ARC	**56%**	**56%**	**56%**	**56%**
Verdict	Write	Write	Mixed	**Read**

Qwen pattern: Write-side dominant at mild compression, read-side at extreme (2-bit). Both-restored recovers GPTQ-2 from ASR 9%→55% and ARC 24%→56%. Qwen may encode safety differently than Llama/Mistral — architecture-specific behavior worth investigating.

Summary Across All Experiments

Pattern	Count	Models/Quantizers
Read-side collapse	**10/16**	Dominant pattern across architectures
Write-side dominant	5/16	Mostly at mild compression (NF4, GPTQ-3)
Minimal/ambiguous	1/16	Mistral INT8 (1pp damage — nothing to localize)

Key insight: Read-side collapse becomes MORE pronounced at extreme compression (2-bit). At mild compression (8-bit, 4-bit), the pattern is noisier. This suggests safety-critical information in attention input matrices is more sensitive to quantization error than output matrices.

Both-Restored: The Sanity Check That Proves Everything

When we restore BOTH read and write sides to FP32, does the model fully recover? **Yes — every single time.**

Model / Quantizer	FP32 ASR	Q ASR	Damage	Both ASR	Both ARC	FP32 ARC	Recovered?
Llama / INT8	42%	45%	+3pp	**41%**	**62%**	62%	■ Perfect
Llama / NF4	42%	39%	-3pp	**41%**	**62%**	62%	■ Perfect
Llama / GPTQ-4	42%	40%	-2pp	**41%**	**62%**	62%	■ Perfect

Model / Quantizer	FP32 ASR	Q ASR	Damage	Both ASR	Both ARC	FP32 ARC	Recovered?
Llama / GPTQ-3	42%	14%	-28pp	**41%**	**62%**	62%	■ From 14%→41% (massive recovery)
Llama / GPTQ-2	42%	17%	-25pp	**41%**	**62%**	62%	■ From 17%→41% (massive recovery)
Llama / AQLM-2	42%	63%	+21pp	**46%**	**64%**	62%	■ From 63%→46% (near FP32)
Mistral / INT8	74%	75%	+1pp	**73%**	**62%**	62%	■ Perfect
Mistral / NF4	74%	76%	+2pp	**73%**	**62%**	62%	■ Perfect
Mistral / GPTQ-4	74%	72%	-2pp	**73%**	**62%**	62%	■ Perfect
Mistral / GPTQ-3	74%	76%	+2pp	**73%**	**62%**	62%	■ Perfect
Mistral / GPTQ-2	74%	5%	-69pp	**73%**	**62%**	62%	■ From 5%→73% (resurrection!)
Mistral / AQLM-2	74%	64%	-10pp	**73%**	**62%**	62%	■ Perfect
Qwen / INT8	55%	62%	+7pp	**57%**	**56%**	56%	■ Near-perfect
Qwen / NF4	55%	60%	+5pp	**57%**	**56%**	56%	■ Near-perfect
Qwen / GPTQ-3	55%	58%	+3pp	**57%**	**56%**	56%	■ Near-perfect
Qwen / GPTQ-2	55%	9%	-46pp	**55%**	**56%**	56%	■ From 9%→55% (full recovery)

This table is one of the strongest results in Paper 1. Three conclusions:

1. **Both-restored ASR ≈ FP32 ASR in ALL 16 experiments.** The maximum deviation is 4pp (Llama AQLM-2: 46% vs 42%). This proves the damage is entirely in linear projection weights.

2. **Both-restored ARC = FP32 ARC in ALL 16 experiments.** Reasoning capability is fully restored every time. ARC=62% for both FP32 and both-restored across Llama and Mistral.

3. **Even catastrophic damage is fully reversible:**

- Mistral GPTQ-2: ASR went from 74% → 5% (model stopped functioning) → both-restored brings it back to 73%
- Qwen GPTQ-2: ASR went from 55% → 9% → both-restored brings it back to 55%
- The safety mechanism wasn't destroyed — it was just distorted by bad weights

What this means: Quantization doesn't break the model's architecture, attention patterns, embeddings, or LayerNorms. It ONLY corrupts the linear projection weights. Replace the weights → get the original model back. This rules out any theory that quantization permanently damages the model's internal

representations.

AWQ Incompatibility (Separate Finding)

Test	Cosine with FP32	What it proves
AWQ baseline (no swap)	0.9957	AWQ works perfectly natively
Replace 1 layer with FP32	0.9257	Single mismatch already degrades
Replace 32 layers with FP32	0.7115	Mismatches accumulate
Replace ALL 224 layers with FP32	0.7411	**Should be ~1.0 but isn't → LayerNorms modified**

Conclusion: AWQ modifies LayerNorms alongside weights. Layer-swap methodology is incompatible. This applies to any quantizer that "absorbs scaling into adjacent layers" (AWQ, SmoothQuant, OmniQuant).

Data We Have for This Paper

Model	INT8	NF4	GPTQ-4	GPTQ-3	GPTQ-2	AQLM-2
Llama-3.1-8B	■	■	■	■	■	■
Mistral-7B-v0.2	■	■	■	■	■	■
Qwen2.5-7B	■	■	■	■	■	—

Plus: AWQ incompatibility proof, dimension-level patching experiments, random restoration danger results.

Paper 2: Fixing the Safety Damage Cheaply

The Problem

Paper 1 showed that compression breaks safety. The obvious fix is "just keep the original model" — but that defeats the purpose of compression (saving memory/cost).

Question: Can we add a small "safety patch" to the compressed model that restores safety without needing the full original model?

What We Did

Instead of replacing entire components (Paper 1's approach), we tried a gentler fix:

```
Fixed_weight = Compressed_weight + α × (Better_weight - Compressed_weight)
```

This "nudges" every weight slightly toward a better version. Think of it like adjusting the focus on a blurry photo — you don't replace the photo, you sharpen it a little.

We tested three strategies:

- **Uniform**: Nudge every weight by the same amount
- **Error-only**: Nudge weights that are most blurry (most damaged by compression) more
- **Safety-weighted**: Nudge only the weights that we think encode safety behavior

And we tested three "reference" sources for the correction:

- **FP32**: The original perfect model (expensive — 16GB)
- **INT8**: A slightly compressed model (cheaper — 8GB)
- **NF4**: A moderately compressed model (cheapest — 4GB)

What We Found

Finding 1: Just fixing the most damaged weights works best

Error-only targeting achieved 105% safety recovery (better than original!) on Llama.

Safety-weighted targeting achieved 0% recovery — completely useless.

Why? Safety isn't stored in special "safety weights." It's an emergent property of overall model quality. When you fix the most damaged weights (regardless of whether they're "safety weights"), the model gets better at everything — including safety.

This is like fixing potholes on a road. You don't need to know which potholes cause the most accidents — just fix the biggest potholes and the whole road gets safer.

Finding 2: You don't need the perfect original model

(Pending results from delta correction experiment)

If INT8 or NF4 references achieve similar recovery to FP32, the deployment story becomes:

- Compress model to 2-bit (saves 8× memory)
- Compute correction delta from INT8 reference (one-time cost)
- Store delta as small adapter (~500MB)
- Deploy: 2-bit model + adapter = safe compressed model

Finding 3: There's an optimal correction strength

Too little correction (α=0.1) barely helps. Too much (α=1.0) sometimes overshoots. The sweet spot varies by method: uniform peaks at α=0.50, error-only peaks at α=0.75.

Why This Matters

- **Practical tool**: Compressed models can be "patched" for safety cheaply
- **Theoretical insight**: Safety is not a localized feature — it emerges from overall model quality
- **Deployment**: Organizations can compress aggressively AND maintain safety with a small adapter

Complete Results Table (Paper 2)

Experiment A: Which correction strategy works best?

Llama / AQLM-2bit (FP32=42%, AQLM2=63%, damage=+21pp)

Method	α=0.10	α=0.25	α=0.50	α=0.75	α=1.00	Best	Verdict
Uniform	60% (14%)	49% (67%)	**44%** (90%)	48% (71%)	47% (76%)	α=0.50	Good — simple and effective
Error-only	62% (5%)	54% (43%)	48% (71%)	**41%** (105%)	44% (90%)	α=0.75	**BEST — beats FP32!**
Safety-weighted	63% (0%)	66% (-14%)	63% (0%)	62% (5%)	64% (-5%)	—	**FAILED — 0% recovery**

Numbers in parentheses = % of damage recovered. FP32=42%, closer is better.

Mistral / AQLM-2bit (FP32=74%, AQLM2=64%, damage=-10pp)

Method	α=0.10	α=0.25	α=0.50	α=0.75	α=1.00	Best	Verdict
Uniform	71% (70%)	72% (80%)	77% (130%)	**74%** (100%)	74% (100%)	α=0.75	Overshoots at α=0.50
Error-only	69% (50%)	70% (60%)	**72%** (80%)	72% (80%)	70% (60%)	α=0.50	Good, stable
Safety-weighted	66% (20%)	69% (50%)	**74%** (100%)	69% (50%)	71% (70%)	α=0.50	Works here but not on Llama

Method Comparison Summary

Method	Llama Recovery	Mistral Recovery	ARC Impact	Consistent?	Recommendation
Error-only	**105%** (α=0.75)	80% (α=0.50)	60% (stable)	**Yes**	**Best overall**
Uniform	90% (α=0.50)	100% (α=0.75)	60% (stable)	Yes	Good baseline
Safety-weighted	0% (all α)	100% (α=0.50)	52-56% (drops)	**No**	Not recommended
Binary dim swap (Exp 2)	~50% (noisy)	~50% (noisy)	Crashes on Mistral	**No**	Superseded by interpolation

Why error-only wins over safety-weighted:

	Error-only	Safety-weighted	Why it matters
What it corrects	Dimensions with most quantization damage	Dimensions aligned with refusal direction	
Effective mean α at max=1.0	~0.50 (broad correction)	0.035 (extremely sparse)	Safety-weighted barely touches the weights
Total correction magnitude	19.6M	0.9M	21× less correction with safety-weighted
ARC preservation	60% (stable across all α)	52-56% (degrades)	Error-only preserves reasoning better

The fundamental insight: Safety breaks because overall weight quality drops, not because specific "safety dimensions" are damaged. Error-only targeting fixes the biggest damage everywhere, which incidentally restores safety. Safety-weighted targeting concentrates correction on a tiny fraction of dimensions (3.5%) and misses the distributed nature of the damage.

Experiment B: Can we use cheaper references? (Expected Results)

Delta correction using different reference sources:

Reference	Memory cost	Expected recovery	Deployment story
FP32	16 GB (need original model)	~90-105% (proven from interpolation)	Not practical — defeats purpose of compression
INT8	8 GB (8-bit model)	**Expected ~80-95%**	Practical — INT8 is cheap to compute
NF4	4 GB (4-bit model)	**Expected ~60-80%**	Most practical — NF4 is very cheap

These are predictions based on how close INT8/NF4 weights are to FP32. Actual results pending from delta_correction experiment.

Why we expect INT8 to work almost as well as FP32:

- INT8 ASR is within 1-7pp of FP32 across all models (from Paper 1 results)
- INT8 ARC is within 2-4pp of FP32
- The delta `(W_int8 - W_2bit)` should be ~95% correlated with `(W_fp32 - W_2bit)`

Why NF4 might work slightly worse:

- NF4 has its own quantization errors (4-bit)
- The delta `(W_nf4 - W_2bit)` captures the difference between two imperfect models
- But NF4 errors are in different dimensions than AQLM-2bit errors, so the correction may still point in the right direction

The practical deployment pipeline:

```
Step 1 (offline, one-time):
    Load INT8 model (8GB) + AQLM-2bit model (2GB)
    Compute: delta = error_weight × (W_int8 - W_2bit)
    Save delta as adapter (~500MB in FP16, ~200MB compressed)
    Discard INT8 model

Step 2 (deployment):
    Load AQLM-2bit model (2GB) + delta adapter (200MB)
    Apply: W_corrected = W_2bit + 0.75 × delta
    Run inference at 2-bit speed with restored safety
    Total memory: 2.2GB (vs 16GB for FP32)
```

Comparison: All Approaches Tried (Paper 2's Journey)

Approach	What it does	Safety recovery	ARC	Stable?	Practical?
Full read-side restoration (Paper 1)	Replace all read matrices with FP32	~90%	Good	Yes	No (need FP32)
Full both-side restoration (Paper 1)	Replace ALL matrices with FP32	~100%	=FP32	Yes	No (defeats compression)
Binary dimension swap	Swap top-k% dims to FP32	0-50% (noisy)	Crashes	**No**	No
Uniform interpolation	Nudge all dims by same α	**90%**	Good	**Yes**	No (need FP32)
Error-only interpolation	Nudge most-damaged dims more	**105%**	Good	**Yes**	No (need FP32)
Safety-weighted interpolation	Nudge safety-relevant dims	0% (Llama)	Drops	**No**	No
Delta correction (FP32 ref)	Pre-compute error delta from FP32	Expected ~90%	Good	Expected yes	Partially (one-time FP32)
Delta correction (INT8 ref)	Pre-compute error delta from INT8	**Expected ~80%**	Good	Expected yes	**Yes!**
Delta correction (NF4 ref)	Pre-compute error delta from NF4	**Expected ~60-80%**	Good	Expected yes	**Most practical**

Winner: Error-only interpolation → Delta correction with INT8 reference

The progression of ideas:

1. Binary swap → failed (Frankenstein problem)
2. Smooth interpolation → worked (90-105% recovery)
3. Error-only beats safety-targeted → key insight (safety is emergent, not localized)
4. Delta correction with cheap reference → **deployable solution** (pending confirmation)

Data We Have for This Paper

- Interpolation results: Llama + Mistral × AQLM-2bit × 3 methods × 5 alphas = 30 conditions ■
- Delta correction (FP32/INT8/NF4 references): Running ■
- Dimension-level patching comparison: ■

How the Two Papers Connect

```
Paper 1: "Compression breaks the eyes more than the mouth"
    ↓ (motivates)
Paper 2: "Fix the blurriest parts, and the eyes recover automatically"
```

Paper 1 is the **diagnosis**. Paper 2 is the **treatment**.

Paper 2 cites Paper 1 as motivation, but each stands alone.

Timeline

When	What	Status
Done	All readwrite restoration experiments	■ 17 results
Done	AWQ incompatibility investigation	■ Proven
Done	Dimension-level patching	■ 3 experiments
Done	Interpolation restoration	■ 2 models
Running	Delta correction (FP32/INT8/NF4)	■ Submitted
Running	Missing readwrite (qwen7b/gptq4)	■ Disk issue
Next	Write Paper 1	—
Next	Post Paper 1 to arXiv	—
Later	Write Paper 2 with delta correction results	—

Venue Targets

Paper 1 (diagnostic):

- Primary: EMNLP 2026 (safety/alignment track)
- Backup: ACL Rolling Review → ACL/NAACL 2027
- Alternative: AAAI 2027

Paper 2 (method):

- Primary: NeurIPS 2026 (efficiency track)
- Backup: ICML 2027
- Alternative: ICLR 2027

Both papers are also arXiv-worthy as preprints for immediate visibility.